
EVE ROBOTICS PROJECT

Web GUI & ROS 2 Control System

A Technical Reference and Source Code Analysis

Target Robot System:	EVE (Self-Stabilizing Two-Wheeled Robot)
Author/Maintainer:	Networks Project GUI Development Team
OS/Platform:	Ubuntu Server / ROS 2 Humble / Raspberry Pi 4
Created/Updated:	May 2026
Status:	Fully Operational (QoS & Type-Aligned)

1. System Architecture & Overview

The **EVE Web GUI** serves as the primary ground station control panel and telemetry monitor for the EVE two-wheeled robot. The codebase is adapted from an older multi-camera ROV system ('Triton') and modernized into a highly optimized, single-camera, self-stabilizing wheeled vehicle controller.

The platform is composed of three decoupled architectural layers:

- **Front-End Browser Layer:** A clean, HTML5 + CSS3 dashboard containing a single camera Canvas, dynamic PlayStation controller tester UI, live sensor indicators, and a persistent QR code history manager.
- **Local Server Layer (Flask/Socket.IO):** Written in Python, it coordinates high-speed gamepad sampling and live sensor broadcasts. It utilizes a long-polling Socket.IO transport and runs on a secure background thread inside a standard ROS 2 Node.
- **Robot Control Layer (ROS 2):** Runs on the Raspberry Pi 4, handling raw sensor subscriptions (ultrasonics, IMU) and sending high-frequency actuator directives via `/cmd_vel`.

Important Note: Network Architecture

Because the robot operates in remote environments, a *Cloudflare Tunnel* is used to expose the localhost ports of the Raspberry Pi 4 securely to the web. This allows zero-latency control via Socket.IO and WebSocket streams without exposing port forwards to the open internet.

2. Deep Dive: Back-End ROS 2 Flask Node (node.py)

The core orchestration of the back-end takes place in `node.py`. This script defines the `EVEGUINode` class, inheriting from `rcipy.node.Node`. It seamlessly bridges concurrency between synchronous ROS 2 event loop threads and asynchronous Socket.IO connections.

Key Core Components in node.py:

- **Multithreaded Execution:** Since `rcipy` and `Flask` are both blocking, the script initializes the `Flask-SocketIO` app in the primary thread, while launching a dedicated background Python thread (`__ros_thread`) to run the `rcipy.spin()` function.
- **QoS Type Alignment & Topic Subscriptions:** Subscribes to the status topic `/status`. It is strictly configured to use `custom_msgs.msg.Status`. This aligns with the exact ROS 2 package compiled on the robot. Previously, a type mismatch with `eve_control_types` blocked all incoming messages.
- **Command Velocity (/cmd_vel):** Computes twist commands at 5Hz using physical joystick inputs received from the front-end browser, publishing standard `geometry_msgs/msg/Twist` messages.

Code Snippet: Subscription and Status Callback in EVEGUINode

```
# Subscriber Initialization
self.create_subscription(Status, '/status', self.__status_callback, self.__qos_profile())

# Callback Handler delivering data to Socket.IO
def __status_callback(self, msg: Status):
    socketio.emit('robot_status', {
        'distance_cm': [
            int(msg.distance_cm[0]), # Left sensor
            int(msg.distance_cm[1]) # Right sensor
        ],
        'euler': {
            'x': float(msg.euler.x),
            'y': float(msg.euler.y),
            'z': float(msg.euler.z)
        }
    })
```

3. Deep Dive: Front-End Client (script.js)

The client-side interface is managed by script.js, built with robust vanilla JS for optimal execution speed. It handles Socket.IO socket connections, maps PlayStation controller inputs, draws high-speed camera frames to an HTML5 canvas, and operates a Computer Vision QR code scanner.

Ultrasonic Sensor Data Display:

The EVE robot is equipped with two lateral ultrasonic sensors (Left and Right). Because the custom message features a 2-element array (uint8[2] distance_cm), script.js acts on the incoming robot_status Socket.IO event and maps the elements as follows:

- distance_cm[0] maps to the **Left** panel value (#diag-left-val).
- distance_cm[1] maps to the **Right** panel value (#diag-right-val).
- The **Front** and **Back** telemetry indicators stay populated with — as they are not wired, preventing index errors.

Computer Vision QR Code Scanner:

Integrating local camera frame parsing using the **jsQR** library, the GUI acts as an offline scanner without adding CPU overhead to the Raspberry Pi 4:

- **Throttled Frame Analysis:** Every time a new frame is painted to the canvas, scanForQrCode() is called. It uses a 400ms throttle to prevent unnecessary UI lagging.
- **Local Processing:** Extracts pixel data using canvas.getContext('2d').getImageData() and passes the raw buffer to jsQR().
- **Robust Data Storage:** When a code is detected, it is saved inside the browser's localStorage with a readable date and unix timestamp. It blocks rapid duplicate scans of the same code within 5 seconds.

Code Snippet: Canvas Image Extraction and jsQR Decoding

```
function scanForQrCode() {
  if (!DOM.cameraCanvas || !window.jsQR) return;
  var now = Date.now();
  if (now - state.lastQrScanTime < CONFIG.QR_SCAN_INTERVAL) return;
  state.lastQrScanTime = now;
  try {
    var w = DOM.cameraCanvas.width;
    var h = DOM.cameraCanvas.height;
    if (w < 10 || h < 10) return;
    var ctx = DOM.cameraCanvas.getContext('2d');
    var imageData = ctx.getImageData(0, 0, w, h);
    var code = jsQR(imageData.data, w, h, { inversionAttempts: 'dontInvert' });
    if (code && code.data) {
      addQrCode(code.data);
    }
  } catch (e) {}
}
```

4. Deep Dive: Offline Mock Framework (run_mock.py)

Developing on ROS 2 systems often requires working locally on Windows or Mac machines that do not have ROS 2 libraries installed. To bypass these dependencies during development, run_mock.py serves as an independent mockup container.

How the Mock Framework Works:

Before importing eve_web_gui.node, the script injects empty mock modules into Python's system module table. This overrides any imports that would normally crash due to missing shared libraries:

- **sys.modules Injections:** Intercepts all references to rclpy, geometry_msgs, and custom_msgs, routing them to dynamic mock instances.
- **Mock classes:** Provides functional placeholders for ROS nodes, publishers, subscribers, client objects, and QoS configurations.
- **Clean Standalone Server:** Allows the Flask webserver and WebSocket engines to run exactly like they would on the robot, providing live feedback for styling and UI testing.

Code Snippet: ROS 2 Module Hijacking in run_mock.py

```
class MockModule:
    pass

mock_custom_msgs = MockModule()
mock_custom_msgs_msg = MockModule()
mock_custom_msgs_msg.Status = MockModule() # Mocking EVE Status Type

sys.modules['rclpy'] = MockModule()
sys.modules['rclpy.node'] = MockModule()
sys.modules['custom_msgs'] = mock_custom_msgs
sys.modules['custom_msgs.msg'] = mock_custom_msgs_msg

# Safe local boot of the main node entry-point
from eve_web_gui.node import main
```

5. Summary of Recent Improvements

By identifying and resolving the type mismatch between eve_control_types and custom_msgs, and narrowing down the index parsing from 4 to 2 elements, the **EVE Web GUI** now starts, subscribes, and communicates faultlessly with the robot's physical sensors. Combined with the offline QR scanner and high-performance glassmorphism UI, the EVE dashboard stands as a premium, robust human-machine interface.